```
http = require 'http'

extractData = (topic, callback) ->
  options =
    host: 'www.agtronsemporium.com'
    port: 80
    path: "/data/#{topic}"

  http.get(options, (res) ->
    callback res
  ).on 'error', (e) ->
    console.log e
```

Spot the dependency.

As you can see, the `http` dependency makes testing very difficult because it breaks all the tests any time the external site is offline. You can solve this with an `http` double. The double must know how to act like the real thing, so by looking at `extractData` you can see that the double must have a `get` method and an `on` method:

```
http =
  get: (options, callback) ->
    callback 'canned response'
    @
  on: (event, callback) ->
    # do nothing
```

Return the current object to create a fluent interface (see chapter 7).

To the `extractData` function, this `http` double will look just like a real `http`. In duck-typing terms (chapter 7), it walks like an `http` and quacks like an `http`, so it's an `http`.

Having the double is only half the story; it also needs to be injected during the test. In order to allow this, the `extractData` function must be changed to allow the dependency to be injected: